University of Geneva

Faculty of Science

My Course

Digital Forensics

14x065

Michel Jean Joseph Donnet

2026-03-12

Contents

1 Introduction

Humans have always tried to capture the present moment, both in painting and in photography. However, the photos taken are not always of good quality, that's why some computational methods, such as image filtering, have been developped to increase the quality of a given picture.

In this project, we will examine some basic image filtering methods and some basic way to reconstruct damaged pictures.

2 Methodology

The image filtering works in both spatial and Fourier domain.

The basics are the following:

- A filter is created to apply some operation on the image. For instance, a laplacian filter gives the gradient of the image.
- The filter is applied to the image in spatial or in Fourier domain.
  - In spatial domain, the convolution between the image and the filter is performed to apply the filter to each pixel of the image.
  - In Fourier domain, a simple multiplication between the image and the filter applies the filter to the image.

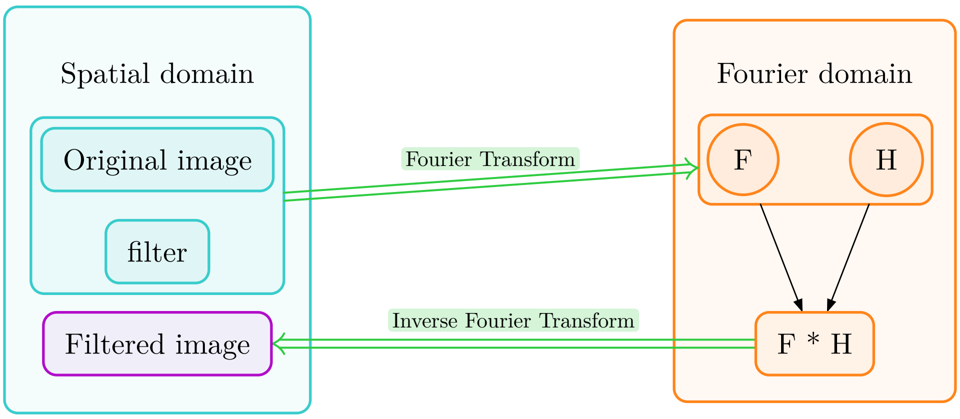


Figure 1: Image filtering in Fourier domain

2.1 Types of filters

In this project, we will use the following filters:

- Gaussian Blur: a low-pass filter that blurs the image by reducing the high-frequencies components. In this way, the small details (contained in high frequencies) are removed.
- Unsharp Mask: a high-pass filter that sharpens the image by enhancing the high-frequency components. As the low-pass filter kill the high frequencies, the unsharp mask can be obtained by substracting a blurred version of the image to the original image. Indeed,

by substracting to the original image an image composed of low frequencies, only high frequencies are kept:

$$\text{Unsharpened image} = \text{Image} - \text{Blurred image}$$

2.2 Image restoration

2.2.1 Inverse filtering

Inverse filtering is a technique used to restore an image that has been filtered by a known filter. As the filtering operation in Fourier domain is a simple multiplication, the inverse filter consists only to inverse the multiplication to retrieve the original image For instance, if the image was filtered in Fourier domain by a known filter H, the original image can be retrieved by multiplying the filtered image by the inverse filter $\frac{1}{H}$. Note that in general, to avoid division by zero, a small value K is added to stabilize the inverse filter:

$$\text{Inverse filter} = \frac{1}{H+K}$$

2.2.2 Wiener filtering

The wiener filter is based on the inverse filter method. However, the wiener filter takes care about the noise present in the image. Indeed, the inverse filter amplify the noise, and the wiener filter allows to avoid the noise by adding a term dependent on the noise present in the image. The wiener filter is defined as follows:

$$\text{Wiener filter} = \frac{1}{H} \cdot \left(\frac{|H|^2}{|H|^2 + \frac{1}{\text{SNR}}} \right)$$

with $\text{SNR} = \frac{\text{Average pixel value in noised image}}{\sigma_{\text{noise}}}$. To stabilize the wiener filter, a small value K is added like for the inverse filter.

2.2.3 Gradient-based image restoration

We can consider that we have some image b that was filtered by a filter A (Here, we can consider that A is a convolution matrix). We want to retrieve the original image x such that $Ax = b$.

To do that, we can use the gradient descent method to minimize the mean squared error between Ax and b:

$$\arg\min_x f(x) = \arg\min_x \frac{1}{2} \|Ax - b\|_2^2$$

.

So the gradient descent algorithm give us:

$$x^{k+1} = x^k - \alpha \nabla_x f(x^k)$$

The gradient $\nabla_x f$ can be defined as follows:

$$
\begin{aligned}
\nabla_x f(x) &= \nabla_{x(\frac{1}{2}\|Ax-b\|_2^2)} \\
&= \nabla_x \left(\frac{1}{2}(Ax-b)(Ax-b)^T\right)) \\
&= \nabla_x \left(\frac{1}{2}(x^TA^TAx - x^TA^Tb - Axb^T + bb^T)\right) \\
&= \nabla_x \left(\frac{1}{2}(x^TA^TAx - 2x^TA^Tb + bb^T)\right) \\
&= \frac{1}{2}(2A^TAx - 2A^Tb) \\
&= A^TAx - A^Tb
\end{aligned}
$$

In our work, the numnber of iterations is used as stopping criterion for the gradient descent algorithm.

Note that as A is a convolution matrix, the transpose of A is the convolution with the flipped version of the filter.

2.2.3.1 Stochastic gradient descent

The stochastic gradient descent is based on the gradient descent method. However, instead of computing the gradient on all the image, the stochastic gradient descent select a set of rows and columns of the image to compute the gradient on this subset of the image. This variation is faster than the classical gradient descent thanks to the computation on a subset of the image. However, as the gradient is not computed for the whole image, the result is less accurate than the classical gradient descent.

3 Implementation

The code can be executed in command line and follows the documentation below:

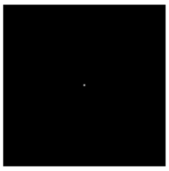
```
TP1 of computational imaging

options:
  -h, --help         show this help message and exit
  -t, --task TASK    Enter the number of the task to execute
  -i, --image IMAGE  Path to the input image
```

For the gradient descent and the stochastic gradient descent, to avoid allocation of hudge convolution matrix, I try to filter the image in Fourier domain. Indeed, if the image is of size 64×64, it means that the flat version of the image is of size 4096 and then the convolution matrix needed is of size $4096 \times 4096 = 16777216$. If we consider that each element of the matrix is an integer of 1 octet, it means that the matrix uses 16 Mo. So due to memory usage, I make another implementation: we know that convolution gives multiplication in Fourier domain. So instead of working with convolution matrix of the filter with the image, I use multiplication in Fourier domain. And the flipped version of the filter gives the conjugate in Fourier domain. So for instance the gradient $\nabla_x f$ becomes $G(Hx - b)$ with G the conjugate of the filter in Fourier domain and H the filter in Fourier domain.

4 Results

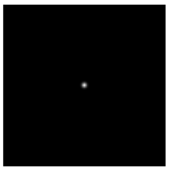
4.1 Image filtering



(a) Gaussian kernel, $\sigma = 0.1$

(b) Spatial domain, $\sigma = 0.1$

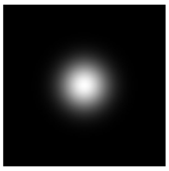
(c) Fourier domain, $\sigma = 0.1$



(d) Gaussian kernel, $\sigma = 1$

(e) Spatial domain, $\sigma = 1$

(f) Fourier domain, $\sigma = 1$

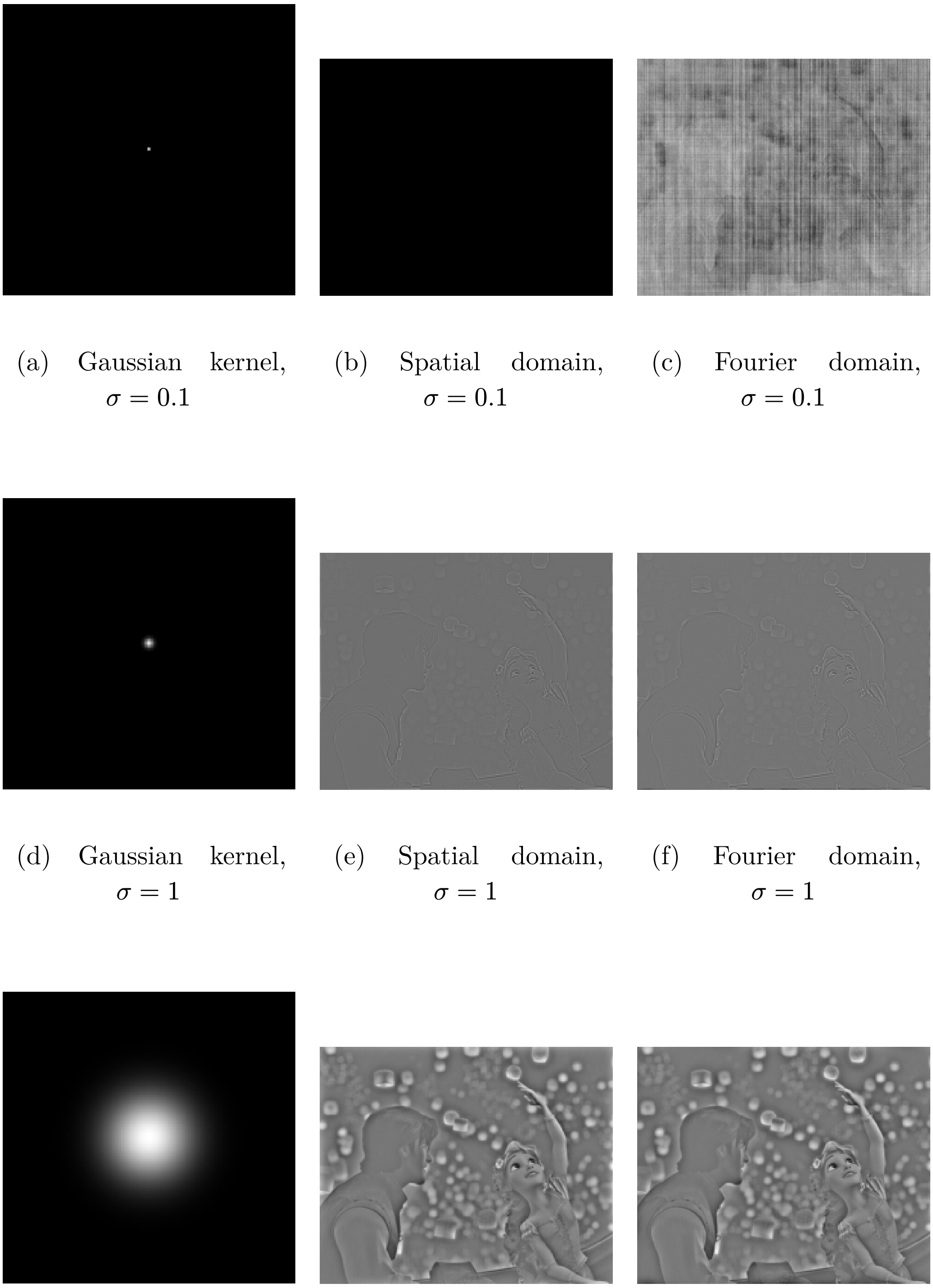


(g) Gaussian kernel, $\sigma = 10$

(h) Spatial domain, $\sigma = 10$

(i) Fourier domain, $\sigma = 10$

Figure 2: Low-Pass filter: Gaussian Blur



(a) Gaussian kernel, $\sigma = 0.1$

(b) Spatial domain, $\sigma = 0.1$

(c) Fourier domain, $\sigma = 0.1$

(d) Gaussian kernel, $\sigma = 1$

(e) Spatial domain, $\sigma = 1$

(f) Fourier domain, $\sigma = 1$

(g) Gaussian kernel, $\sigma = 10$

(h) Spatial domain, $\sigma = 10$

(i) Fourier domain, $\sigma = 10$

Figure 3: High-Pass filter: Unsharp Mask

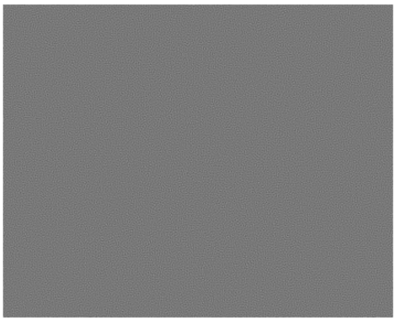
4.2 Image restoration

(a) Original image

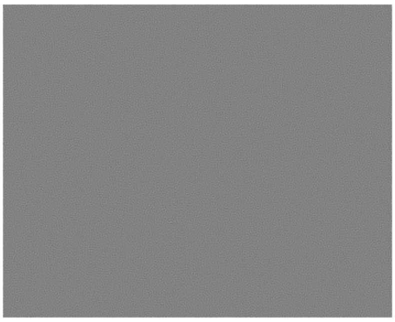
(b) Blured image

Figure 4: Original and blured image

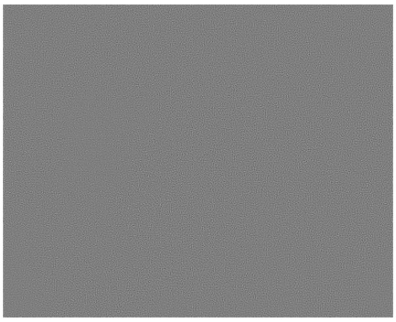
(a) Inverse filtering with noise $\sigma = 0$



(b) Inverse filtering with noise $\sigma = 0.001$



(c) Inverse filtering with noise $\sigma = 0.01$



(d) Inverse filtering with noise $\sigma = 0.1$

Figure 5: Inverse filtering on blured image

(a) Wiener filtering with noise $\sigma = 0$

(b) Wiener filtering with noise $\sigma = 0.001$

(c) Wiener filtering with noise $\sigma = 0.01$

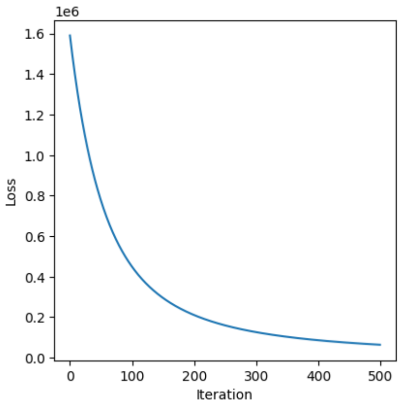
(d) Wiener filtering with noise $\sigma = 0.1$

Figure 6: Wiener filter on blured image

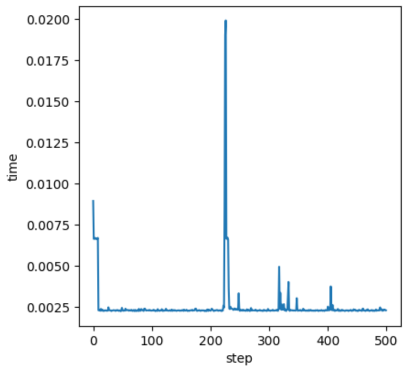
4.3 Gradient-based image restoration

(a) Damaged image

(b) Reconstructed image



(c) Gradient Descent Loss

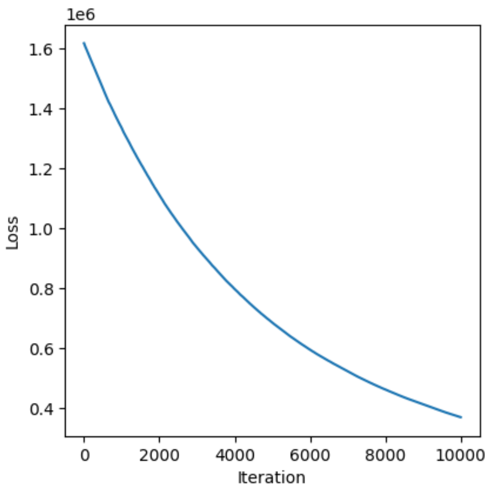


(d) Time taken for gradient descent step

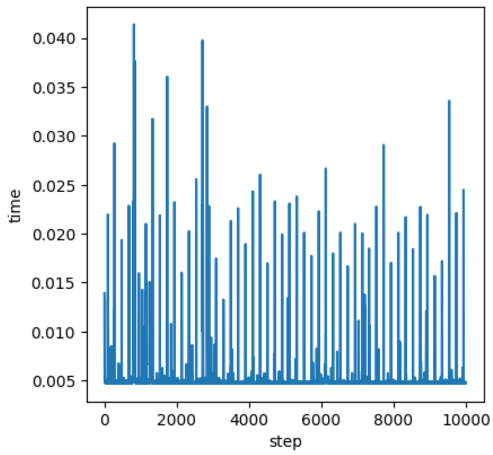
Figure 7: Image restoration using gradient descent in Fourier domain

(a) Damaged image

(b) Reconstructed image



(c) Stochastic Gradient Descent Loss



(d) Time taken for a step

Figure 8: Image restoration using stochastic gradient descent in Fourier domain

```python
print("COucou")
print("Hello")
```

```
COucou
Hello
```

5 Discussion

In the Figure 2, we can easily see that bigger is the Gaussian kernel, more blurred is the image. Indeed, a Gaussian kernel of 0.1 gives an image close to the original image as shown on Figure 2b and Figure 2c whereas a Gaussian kernel of 10 gives a more blurred image as shown in Figure 2h and Figure 2i. So the size of the Gaussian kernel determines the threshold to cut-off high-frequencies.

As shown in Figure 2, the spatial and Fourier domain gives similar results. However, if we compare Figure 2h and Figure 2i, the filtered image in spatial domain have a kind of black blurred border. This is due to the zero padding applied for the convolution of the image with the filter. Indeed, a padding is required in spatial domain to allows the application of the filter on each pixel of the image because of the convolution.

6 Conclusion